Capturing stochastic trends and seasonality with probabilistic forecasting and continual learning

Balaji Ramachandran
balajira@andrew.cmu.edu

Jason Wein
jwein@andrew.cmu.edu

Wendi Zhao
wendizha@andrew.cmu.edu

1 Question and Motivation

Real world dynamical systems often possess three patterns: trend, seasonality/cyclicity and noise. Forecasters aim to accurately capture the part of the system which can be explained, i.e the trend and seasonality. Probabilistic forecasters use stochastic processes to capture systematic noise or stochastic trends in a dynamical system when deterministic forecasters(such as regression) fail to do so. It may happen that dynamical systems alternate between multiple trends during different periodic phases of time. For efficiently capturing this seasonality, we might need a lot of training samples or have to repeatedly finetune the models which has been shown to cause catastrophic forgetting. Our project looks at accurate forecasting of dynamical systems from 2 perspectives: **1.** When there is systematic noise in the system such as an SDE, how well does the forecaster capture the same? **2.** How do we frame the problem of efficiently learning seasonal behaviors as a continual learning problem, which aims at using limited past experiences to prevent catastrophic forgetting in forecasters?

A probabilistic forecaster learns the conditional distribution $\rho_c(x_1 \mid x_0)$ of the future state X_1 given the current state $X_0 = x_0$. Conventional conditional diffusion models [Rasul et al., 2021, Kohl et al., 2023, Price et al., 2025] transport a standard Gaussian distribution to $\rho_c(\cdot \mid x_0)$ through an SDE or ODE, with x_0 supplied as an additional input to the score function or drift vector field. In contrast, Chen et al. [2024] initialize the generative process at x_0 and use an SDE to transport the point mass δ_{x_0} to $\rho_c(x_1 \mid x_0)$ at time $t = 1$. The diffusion coefficient can be adjusted after training, together with a corresponding drift correction; within the proposed family, the choice that minimizes the path-space KL divergence between the exact and approximate processes yields a Föllmer process. **We also want to investigate:** With architecture, training budget, and network evaluations per sample (NFE K) fixed, how does the error of the stochastic-interpolant (SI) forecaster compare with conditional diffusion as a function of τ and K, and which choice explains the difference: point-mass base, diffusion coefficient, or increment parameterization?

2 Related Work

2.1 Stochastic Interpolants Probabilistic Forecasting and Conditional Diffusion Models

Diffusion models reverse a noising SDE with a learned score [Ho et al., 2020, Song et al., 2021b], with fewer NFE via DDIM [Song et al., 2021a] or tuned grids [Karras et al., 2022]. Flow matching [Lipman et al., 2023] and stochastic interpolants [Albergo et al., 2023] regress transports between arbitrary distributions, including data-dependent couplings [Albergo et al., 2024] and data-to-data bridges [Liu et al., 2023]. Chen et al. [2024] use a point-mass base and identify the KL-optimal noise with a Föllmer process [Föllmer, 1986]; their Gaussian-base comparisons (video, Navier–Stokes) neither separate discretization from estimation error nor test residual diffusion. Diffusion models also sample between modes [Aithal et al., 2024]; we measure this.

Preprint.

2.2 Continual Learning for Capturing Seasonal Behaviours

Continual learning (CL) studies how models can adapt to sequentially changing data distributions while avoiding catastrophic forgetting of previously learned patterns. Common approaches include regularization methods such as Elastic Weight Consolidation (EWC) Kirkpatrick et al. [2017] and experience replay, which retains a limited memory of past observations for rehearsal during training Rolnick et al. [2019]. Recent work has applied these ideas to time-series forecasting Besnard and Ragot [2024] and autoregressive PDE surrogates under changing physical regimes, where memory-efficient temporal replay can mitigate forgetting Hemati et al. [2026]. For systems with recurring seasonal regimes, these methods offer a way to adapt forecasters to changing dynamics while retaining predictive performance on previous regimes.

3 Approach and Preliminary Progress

3.1 Comparing SI Probabilistic Forecasting and Conditional Diffusion Models

Models. The SI forecaster uses $I_s = (1-s)x_0 + s^2 x_1 + \varepsilon(1-s)W_s$, $s \in [0,1]$, $\varepsilon = 1$, and fits $\hat{b}$ by minimizing $\mathbb{E}|\hat{b}(I_s, x_0, s) - R_s|^2$ with $R_s = -x_0 + 2s\,x_1 - \varepsilon W_s$. Forecasts start at $X_0 = x_0$ with noise $\sigma_s = \varepsilon(1-s)$, or with the Föllmer coefficient (their Eq. 13), here $g_s^F = \varepsilon\sqrt{(1-s)(3-s)}$ with drift $b^{\mathrm{F}} = \hat{b}\,(3-s)/(2-s) - [2(x-x_0) + s\,x_0]/[s(2-s)]$. The baseline is a VP diffusion model [Song et al., 2021b] with $\beta(t) = 0.1 + 19.9t$ and ϵ-prediction; increment variants model $(x_1 - x_0)/c$, c = s.d. of increments. All networks are 3×128 SiLU MLPs (Adam, batch 512). **Testbeds and results.** We test on a simple Gaussian mixture model from section 4.1 of Chen et al. [2024]. The preliminary results are shown in Figure 1. We also test on the 2d Navier-Stokes Equations (NSE) from section 4.2 of Chen et al. [2024]. We further adapt their associated code to forecast Burger's equation and test a mixture of NSE and Burger's equation to see the SI forecaster's generalization abilities in Figure 2.

3.2 Continual Learning For PDE surrogates

Traditional neural approaches for learning PDEs(PINNs, DeepONets, FNOs) are trained on a single physical regime and do not generalize well to new regimes. In many real-world applications, the underlying physical system may change over time, leading to a need for continual learning approaches that can adapt to new regimes without forgetting previously learned knowledge or changing neural architectures. We use the continual learning framework of Hemati et al. [2026] as a starting point for studying how neural forecasters can adapt to changing dynamical regimes.

They study autoregressive PDE surrogates trained sequentially on different physical regimes and compare naive fine-tuning with methods such as experience replay and elastic weight consolidation (EWC). Their proposed method, Replay-TS, reduces memory requirements by storing selected target temporal states rather than complete rollout trajectories, allowing a fixed memory budget to cover more past experiences. Motivated by this approach, we investigate whether similar continual learning strategies can help forecasters retain previously learned dynamics when the underlying system changes. In particular, we consider Navier-Stokes and Burgers' equations, as well as mixtures of the two for studying adaptation to changing dynamics.

4 Evaluation and Next Steps

Our initial experiments focus on testing probabilistic forecasters on Gaussian mixtures and PDE regimes, and replay temporal slicing from Hemati et al. [2026] method for learning alternating mixture of PDEs. We plan to extend this investigation by comparing sequential fine-tuning with replay-based training to quantify the trade-off between adaptation to new regimes and retention of previously learned dynamics. In particular we plan to look at replay based training of probabilistic forecasters to alternating sets of gaussian mixtures and PDE forecaster surrogates, deploying continual learning techniques for the same and evaluate the proposed method with fine-tuning of probabilistic forecasters. We will also train both model families (SI probabilistic forecasting and conditional diffusion models) on stochastic PDEs to determine in which settings each performs better.

5 Division of Work

Balaji Ramachandran: Worked on the continual learning framework for PDE surrogates. **Jason Wein:** Worked on the NSE and Burgers experiments. **Wendi Zhao:** Worked on the comparative analysis of SI probabilistic forecasting and conditional diffusion models.

6 AI Use Disclosure

In running some of the experiments, Claude 5.5 Opus was used to modify some of the code. The authors take full responsibility for any results.

7 References & Resources

Our work is available at `https://github.com/BalajiRam7/AI_For_Scientific_Computing_Project.git`.

References

S. K. Aithal, P. Maini, Z. C. Lipton, and J. Z. Kolter. Understanding hallucinations in diffusion models through mode interpolation. In *Advances in Neural Information Processing Systems*, 2024.

M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.

M. S. Albergo, M. Goldstein, N. M. Boffi, R. Ranganath, and E. Vanden-Eijnden. Stochastic interpolants with data-dependent couplings. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Q. Besnard and N. Ragot. Continual learning for time series forecasting: A first survey. *Engineering Proceedings*, 68(1):49, 2024. doi: 10.3390/engproc2024068049.

Y. Chen, M. Goldstein, M. Hua, M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden. Probabilistic forecasting with stochastic interpolants and Föllmer processes. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 6728–6756, 2024.

H. Föllmer. Time reversal on Wiener space. In *Stochastic Processes – Mathematics and Physics*, volume 1158 of *Lecture Notes in Mathematics*, pages 119–129. Springer, 1986.

H. Hemati, B. D. Nguyen, and S. Sandfeld. Continual learning for autoregressive PDE surrogates under evolving physical regimes. *Machine Learning for Computational Science and Engineering*, 2026. doi: 10.1007/s44379-026-00085-9.

J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, 2022.

J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.

G. Kohl, L.-W. Chen, and N. Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *arXiv preprint arXiv:2309.01745*, 2023.

Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

G.-H. Liu, A. Vahdat, D.-A. Huang, E. A. Theodorou, W. Nie, and A. Anandkumar. I^2SB: Image-to-image Schrödinger bridge. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025.

K. Rasul, C. Seward, I. Schuster, and R. Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL `https://arxiv.org/abs/1811.11682`.

J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a.

Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021b.

A Preliminary Results

A.1 The comparative analysis of SI probabilistic forecasting and conditional diffusion models

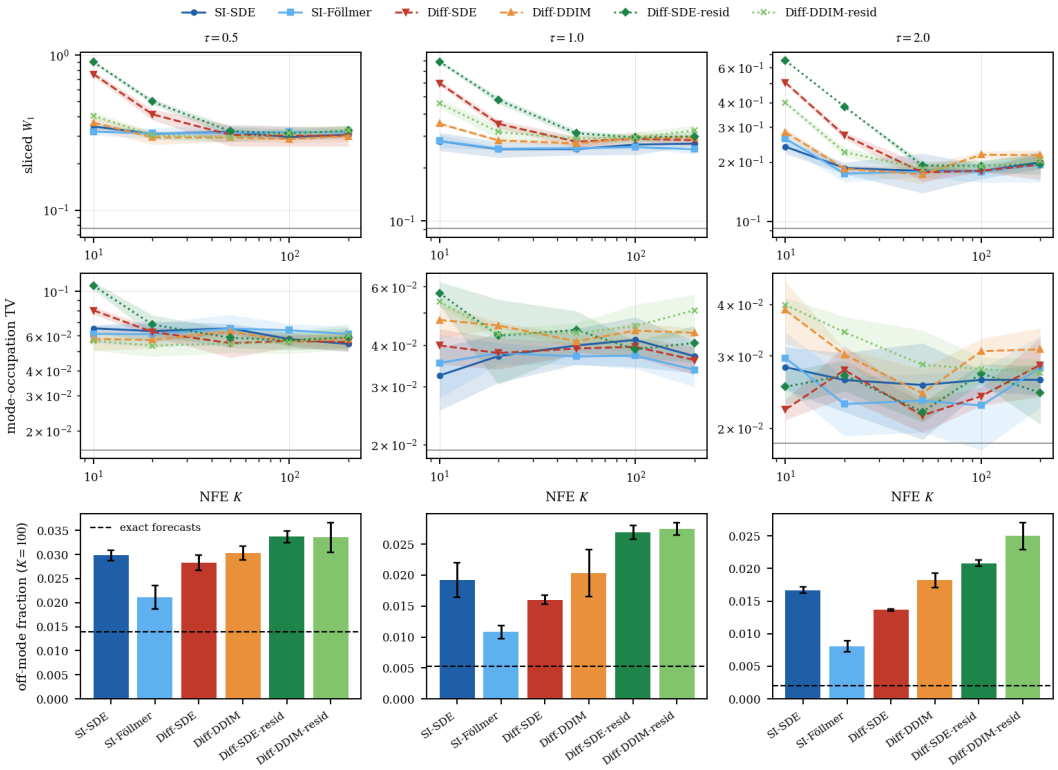


Figure 1: Forecasting the five-mode Gaussian-mixture jump-diffusion of Chen et al. [2024] at lags $\tau \in \{0.5, 1, 2\}$ (columns). All models share one MLP (3×128, SiLU) trained with Adam for 15k steps on 10^5 stationary pairs; every sampler uses K network evaluations (NFE) on a uniform time grid. SI-SDE and SI-Föllmer sample the same learned interpolant drift from $X_0 = x_0$ with noise σ_s or the Föllmer coefficient g_s^{F}; Diff-SDE and Diff-DDIM sample a VP conditional diffusion model of x_1 from Gaussian noise with the reverse SDE or DDIM; the *-resid* variants model the normalized increment $(x_1 - x_0)/c$. Forecasts (2,000 per initial condition) are compared with 2×10^4 simulated paths from four initial conditions: a mode centre, between two modes, near the origin, and an outer tail. **Top:** sliced W_1 distance. **Middle:** total-variation error in the probability of landing in each mode (nearest of five angular sectors). Curves are means over initial conditions; bands show ± 1 s.d. over 3 seeds; gray lines are the Monte Carlo floor, i.e. the score of exact samples of the same size. **Bottom:** fraction of forecasts at $K = 100$ lying between modes; dashed line: exact forecasts; error bars: ± 1 s.d. over seeds.

A.2 Mixed 2D Stochastic Navier-Stokes and Burger

We follow the codebase of Chen et al. [2024] which uses a pseudo-spectral solver with Euler-Maruyama time stepping (we use 200 steps) to solve the PDE. We look at the 2D viscous Burgers equation of the form $\phi_t + \frac{1}{2}|\nabla\phi|^2 = \nu\Delta\phi + F$ with $u = \nabla\phi$ and $\nu = 0.05$. Each system has two trajectories of 100 frames on a 256×256 grid. For Burgers, we replace ω with the divergence $\nabla \cdot u$. In loading the data, each is downsampled to 128×128 and ω_t is reduced to 64×64.

Models are trained on frames 0 to 79 of each trajectory. Evaluation is done on frames $84, 90$ and 96. All models are trained with AdamW with a learning rate $\ell = 2 \times 10^{-4}$ and a batch size of 8.

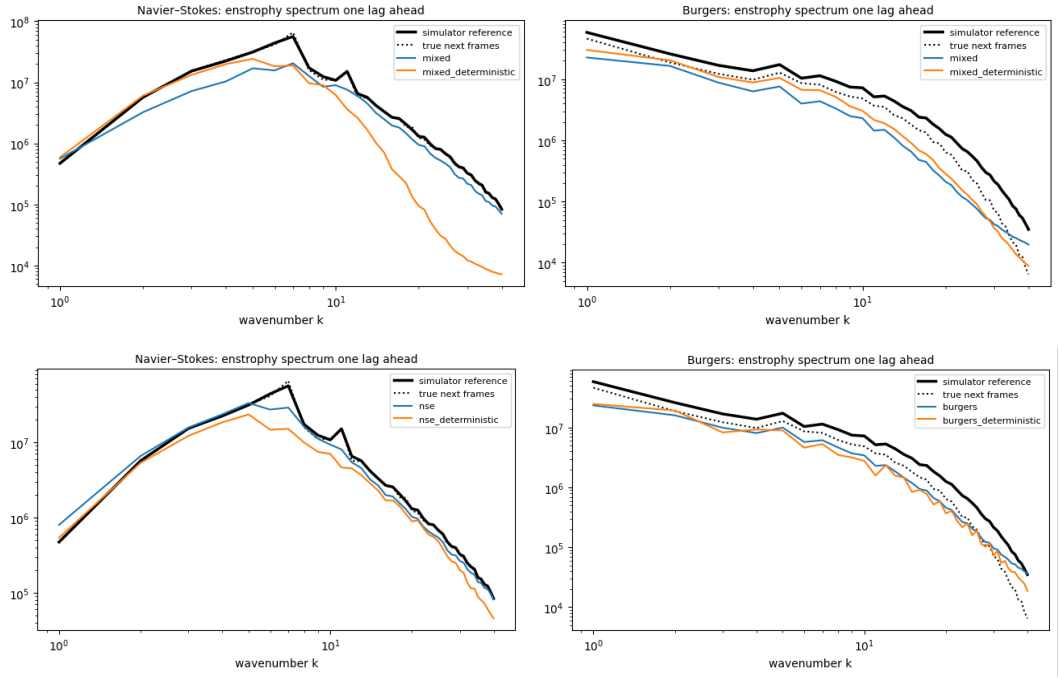


Figure 2: The enstrophy spectra one lag agead ($\tau = 1$) with interpolants $\alpha_s = 1 - s$, $\beta_s = s^2$, $\sigma_s = 1 - s$ and chosen so the diffusion term is $\sigma_s dW_s$. Each is compared to the ground truth and a deterministic version. Over 6 ω_t and 16 independent ensembles of the forecast results $\hat{\omega}_{t+\tau}$ are averaged over to create the spectrum. **Top:** The enstrophy spectra of the model using a dataset with a 50:50 mixture of NSE and Burgers data. **Bottom:** On the left, the enstropy spectrum of the model trained on only NSE and on the right, the enstropy spectrum of the model trained on only Burger's equation. Generally, the performance of the mixed dataset is worse than that of the homogeneous datasets, especially around peaks.

For NS, we consider the random forcing

$$\begin{aligned}\eta_{NS}(t,x,y) =&\big(W_1(t)\sin(6x) + W_2(t)\cos(7x) + W_3(t)\sin(5(x+y)) + W_4(t)\cos(8(x+y))\big)\\ &+ \big(W_5(t)\cos(6x) + W_6(t)\sin(7x) + W_7(t)\cos(5(x+y)) + W_8(t)\sin(8(x+y))\big),\end{aligned} \tag{1}$$

with $\nu = 10^{-3}, \alpha = 0.1, \varepsilon = 1$ and $W_i(t), 1 \le i \le 8$ are independent Wiener processes. For Burger, we consider the random forcing

$$\begin{aligned}\eta_B(t,x,y) =&\big(W_1(t)\sin(x) + W_2(t)\cos(x) + W_3(t)\sin(y) + W_4(t)\cos(y)\big)\\ &+ \big(W_5(t)\sin(x+y) + W_6(t)\cos(x+y) + W_7(t)\sin(x-y + W_8(t)\cos(x-y)\big).\end{aligned} \tag{2}$$

We create four datasets. One has purely NS data, one with purely Burger data, and two with a mixture of the data. We compare these with a deterministic approach trained using a UNet architecture $\hat{\omega}_{t+\tau} = f(\omega_t; \theta)$ trained using Mean Squared Error (MSE) loss function $\mathbb{E}\|\hat{\omega}_{t+\tau} - \omega_{t+\tau}\|^2$ with $\tau = 1$. This comparison is done using the estimated mean and std $\hat{m}, \hat{\sigma}$

$$\text{err(mean)} = \frac{\|\hat{m} - m\|_{L^2}}{\|m\|_{L^2}}, \quad \text{err(std)} = \frac{\|\hat{\sigma} - \sigma\|_{L^2}}{\|\sigma\|_{L^2}}.$$

Let $\hat{\omega}$ be the Fourier coefficients of ω. We define the enstrophy of the vorticity field ω as

$$\text{Enstrophy}(k) = \sum_{k \le |m| \le k+1} |\hat{\omega}(m)|^2.$$

Table 1: Comparisons between the SDE approach and deterministic approach across the different datasets are shown of the short-time and long-time relative accuracy and the long-time of enstrophy. For the long-time row, the models are iterated for 20 lags $\tau = 1$ from 6 held-out starts. The SDE model for Navier-Stokes does worse in the mixed sets, though Burgers performs slightly better with a 50:50 mixture.

	Homogeneous		Mixed 90:10		Mixed 50:50	
Relative error	SDE	Det.	SDE	Det.	SDE	Det.
Navier–Stokes						
Short-time L^2 error of mean	6.2e-1	6.9e-1	6.5e-1	6.9e-1	7.7e-1	6.9e-1
Short-time L^2 error of std	3.5e-1	N/A	6.3e-1	N/A	3.0e-1	N/A
Long-time error of avg. total enstrophy	3.6e2	6.7e-1	6.1e0	7.0e-1	4.3e-1	6.3e-1
Burgers						
Short-time L^2 error of mean	9.9e-1	1.3e0	1.5e0	9.7e-1	8.8e-1	1.1e0
Short-time L^2 error of std	6.3e-1	N/A	7.5e-1	N/A	5.8e-1	N/A
Long-time error of avg. total enstrophy	8.6e-1	7.2e-1	4.4e0	8.6e-1	1.8e-1	8.3e-1

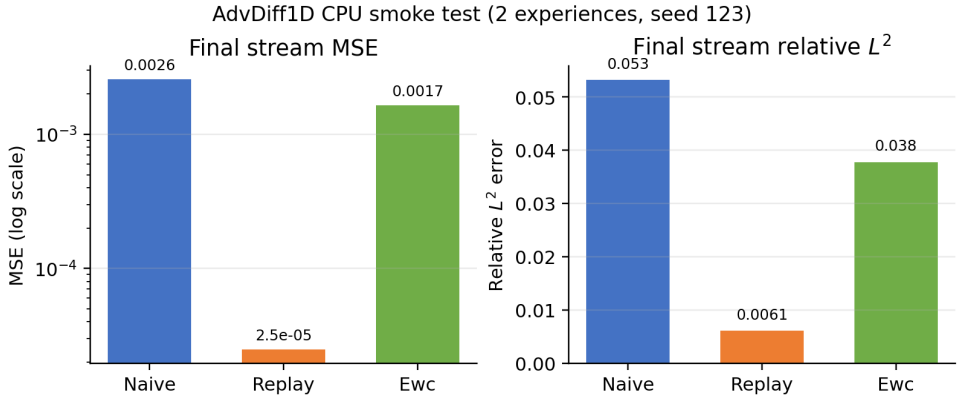


Figure 3: Comparison on the AdvDiff1D stream. Lower values are better. Replay achieves the lowest final stream MSE and relative L^2 error in this fast diagnostic run.

B CPU Smoke-Test Comparison for Continual Learning

To quickly compare continual-learning strategies before committing to the full benchmark, we ran a representative AdvDiff1D smoke test on CPU. The smoke test used two stream experiences, 16 snapshots per simulation, context length 3, rollout length 2, one training epoch, batch size 32, and seed 123. All methods used the same randomly initialized FNO1D model and were evaluated on the same held-out stream. These results are diagnostic only and are not final benchmark results.

Table 2 reports the final metrics. The complete reduced configuration and reproducible runner are maintained with the experiment artifacts; the smoke-test outputs are kept separate from the final benchmark outputs.

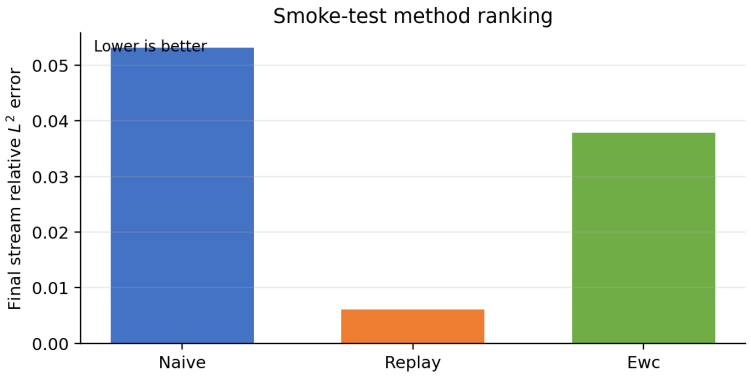


Figure 4: Final stream relative L^2 error for the three smoke-test methods.

Table 2: Final metrics from the AdvDiff1D CPU smoke test.

Method	Stream MSE	Relative L^2
Naive	0.002567	0.05318
Replay	0.0000248	0.00611
EWC	0.001650	0.03780